

## QUERIES BROWSER GROUP 4



### Use Case 1: System Login

Executes when the pharmacist attempts to log into the system using their credentials.

//Validate the pharmacy based on the provided registration number.

**SELECT** id, address, phone, municipality\_id

**FROM** Pharmacy

**WHERE** registration\_number = '28-82016';

### Use Case 2: Search for a Specific Medication

Executes when searching for a medication name to display availability, supplier, and related details across pharmacies.

//Retrieve comprehensive medication data using JOINS across 5 tables.

**SELECT** m.name **AS** Medication\_Name, m.target\_illness, m.SS, m.receta,

p.address **AS** Pharmacy\_Address, p.phone **AS** Pharmacy\_Phone,

mun.name **AS** Municipality,

s.name **AS** Supplier\_Name,

i.stock\_quantity, i.price

**FROM** Medication m

**JOIN** Inventory i **ON** m.id = i.medication\_id

**JOIN** Pharmacy p **ON** i.pharmacy\_id = p.id

**JOIN** Municipality mun **ON** p.municipality\_id = mun.id

**JOIN** Supplier s **ON** i.supplier\_id = s.id

WHERE m.name = 'ibuprofeno';

### Use Case 3: Increase Stock of an Existing Medication

Executes when receiving new units to update the inventory quantities.

//Increment the stock quantity of a specific medication at a specific pharmacy.

**UPDATE** Inventory

**SET** stock\_quantity = stock\_quantity + 50

**WHERE** pharmacy\_id = 'P-0' **AND** medication\_id = 'M-0';

### Use Case 4: Reduce Stock of a Sold Medication

Executes as a transaction when a pharmacist sells a medication to a patient.

//Step 1: Record the sale in the Purchase table.

**INSERT INTO** Purchase (id, patient\_id, pharmacy\_id, date, medication\_id,  
quantity, price)

**VALUES** ('004', 'P-01', 'P-0', '2026-04-15', 'M-0', 1, 5.00);

//Step 2: Deduct the sold amount from the Inventory table.

**UPDATE** Inventory

**SET** stock\_quantity = stock\_quantity - 1

**WHERE** pharmacy\_id = 'P-0' **AND** medication\_id = 'M-0';

### Use Case 5: Add a Non-Registered Medication

Executes when adding a completely new medication to the general registry and its initial inventory

//Step 1: Add the base medication details.

**INSERT INTO** Medication (id, name, target\_illness, SS, receta)

**VALUES** ('M-3', 'amoxicilina', 'antibiotic', 'yes', 'yes');

//Step 2: Add its initial stock to the pharmacy's inventory.

**INSERT INTO** Inventory (id, pharmacy\_id, medication\_id, supplier\_id,  
stock\_quantity, price, expiration\_date, minimum\_stock)

**VALUES** ('I-3', 'P-0', 'M-3', 'S-0', 100, 8.50, '12/2028', 10);

### Use Case 6: Delete an Existing Medication

Executes when removing a medication from the system

//Step 1: Remove inventory records first to clear foreign key constraints.

**DELETE FROM** Inventory

**WHERE** medication\_id = 'M-3';

// Step 2: Remove the medication from the main registry.

**DELETE FROM** Medication

**WHERE** id = 'M-3';

### Use Case 7: Notify Inventory Shortage

Executes periodically or after a sale to check if an alert should be triggered

//Find all medications at a pharmacy where current stock is equal to or below the minimum threshold.

**SELECT** m.name **AS** Medication\_Name, i.stock\_quantity, i.minimum\_stock, p.id  
**AS** Pharmacy\_ID

**FROM** Inventory i

**JOIN** Medication m **ON** i.medication\_id = m.id

**JOIN** Pharmacy p **ON** i.pharmacy\_id = p.id

**WHERE** i.stock\_quantity <= i.minimum\_stock;

### Use Case 8: Order Medication from Supplier to Restock

Executes when generating a purchase order to a supplier.

//Register a new restock order with a status of 'Pending' or 'Sent'.

**INSERT INTO** Orders (id, pharmacy\_id, supplier\_id, medication\_id, date, quantity,  
status)

**VALUES** ('O-3', 'P-0', 'S-1', 'M-1', '2026-04-15', 50, 'Pending');

### Use Case 9: Generate Movement History

Executes when displaying the transaction log of sales and restocks.

//Combine sales (Purchase) and restocks (Orders) into a single chronological view for a specific pharmacy.

```
SELECT date, 'Sale' AS movement_type, medication_id, quantity, price
FROM Purchase
WHERE pharmacy_id = 'P-0'

UNION ALL

SELECT date, 'Restock Order' AS movement_type, medication_id, quantity, NULL
AS price
FROM Orders
WHERE pharmacy_id = 'P-0'

ORDER BY date DESC;
```

### Update Medication Information

Executes when pharmacists update dynamic fields like price, supplier, or expiration dates.

//Update specific details of an existing medication in the inventory.

```
UPDATE Inventory
SET price = 4.50, expiration_date = '05/2028', supplier_id = 'S-1'
WHERE pharmacy_id = 'P-0' AND medication_id = 'M-1';
```

### Basic Retrieval (Adapted from "shows personal information of all donors")

This query simply retrieves all records from the Patient table.

//Query: shows personal information of all patients

```
SELECT * FROM Patient;
```

### Retrieval with Multiple Conditions

Instead of name and surname, we use the patient's name and their unique national ID.

//Query: shows personal information of a patient with a specific name and national ID

```
SELECT * FROM Patient
```

```
WHERE name = 'Paola' AND national_id = '27673831L';
```

### Filtering by Category

Instead of blood type, we search for medications based on the illness they target.

// Query: shows medications for a specific target illness

```
SELECT * FROM Medication
```

```
WHERE target_illness = 'anti-inflammatory';
```

### Grouping and Counting

This counts how many patients reside in each municipality.

// Query: shows how many patients live in each municipality

```
SELECT municipality_id, COUNT(id) AS Total_Patients
```

```
FROM Patient
```

```
GROUP BY municipality_id;
```

### Aggregating Totals

This uses a JOIN and SUM() to calculate the total number of medication units every patient has ever purchased.

//Query: shows the total amount of medication units bought by each patient

```
SELECT p.name, SUM(pur.quantity) AS Total_Medications_Bought
```

```
FROM Patient AS p
```

```
JOIN Purchase AS pur ON p.id = pur.patient_id
```

```
GROUP BY p.id;
```

### Specific Aggregation

This calculates the total money spent by a specific patient by multiplying the price by the quantity.

//Query: shows total amount spent by a specific patient

```
SELECT p.name, SUM(pur.price * pur.quantity) AS Total_Spent
FROM Purchase AS pur
JOIN Patient AS p ON pur.patient_id = p.id
WHERE p.name = 'Alex';
```

### Multi-Table Traversal

This joins three tables to find out exactly which physical pharmacies a specific patient has purchased from.

//Query: shows the pharmacies where a specific patient has made a purchase

```
SELECT DISTINCT ph.address, ph.phone, ph.registration_number
FROM Pharmacy AS ph
JOIN Purchase AS pur ON ph.id = pur.pharmacy_id
JOIN Patient AS p ON pur.patient_id = p.id
WHERE p.name = 'Alba';
```

### Counting Unique Relationships

Using COUNT (DISTINCT ...), this query finds out how many *different* pharmacies each patient visits, preventing duplicate counts if they visit the same pharmacy twice.

//Query: shows how many different pharmacies each patient visits

```
SELECT p.name, COUNT(DISTINCT pur.pharmacy_id) AS
Unique_Pharmacies_Visited
FROM Patient AS p
JOIN Purchase AS pur ON p.id = pur.patient_id
GROUP BY p.id;
```

### Finding "Matches"

Their query matched donors and donees by blood type. This query matches patients to pharmacies that are located in the exact same municipality.

*/\*Query: shows patients and pharmacies that are located in the same municipality*

```
SELECT pat.name AS Patient_Name, ph.registration_number AS Pharmacy_Reg,  
mun.name AS Municipality_Name
```

```
FROM Patient AS pat
```

```
JOIN Municipality AS mun ON pat.municipality_id = mun.id
```

```
JOIN Pharmacy AS ph ON mun.id = ph.municipality_id;
```

### Using Subqueries and ANY

This query finds the supplier details for a specific medication without using traditional JOINS, utilizing the ANY operator and a subquery through the inventory.

*//Query: shows the supplier details for a specific medication using a subquery*

```
SELECT * FROM Supplier
```

```
WHERE id = ANY (
```

```
    SELECT i.supplier_id
```

```
    FROM Inventory AS i
```

```
    JOIN Medication AS m ON i.medication_id = m.id
```

```
    WHERE m.name = 'ibuprofeno'
```

```
);
```

### Advanced Filtering

Since your project heavily features inventory control, this query lists all pharmacies that are completely out of stock of a specific medication

*//Query: shows pharmacies that have zero stock of a specific medication*

```
SELECT p.registration_number, p.address
```

```
FROM Pharmacy AS p
```

```
JOIN Inventory AS i ON p.id = i.pharmacy_id
```

```
JOIN Medication AS m ON i.medication_id = m.id
```

```
WHERE m.name = 'paracetamol' AND i.stock_quantity = 0;
```